Mapeamento Arquitetural de Funções do Jogo e Subsistemas (Flashcards)

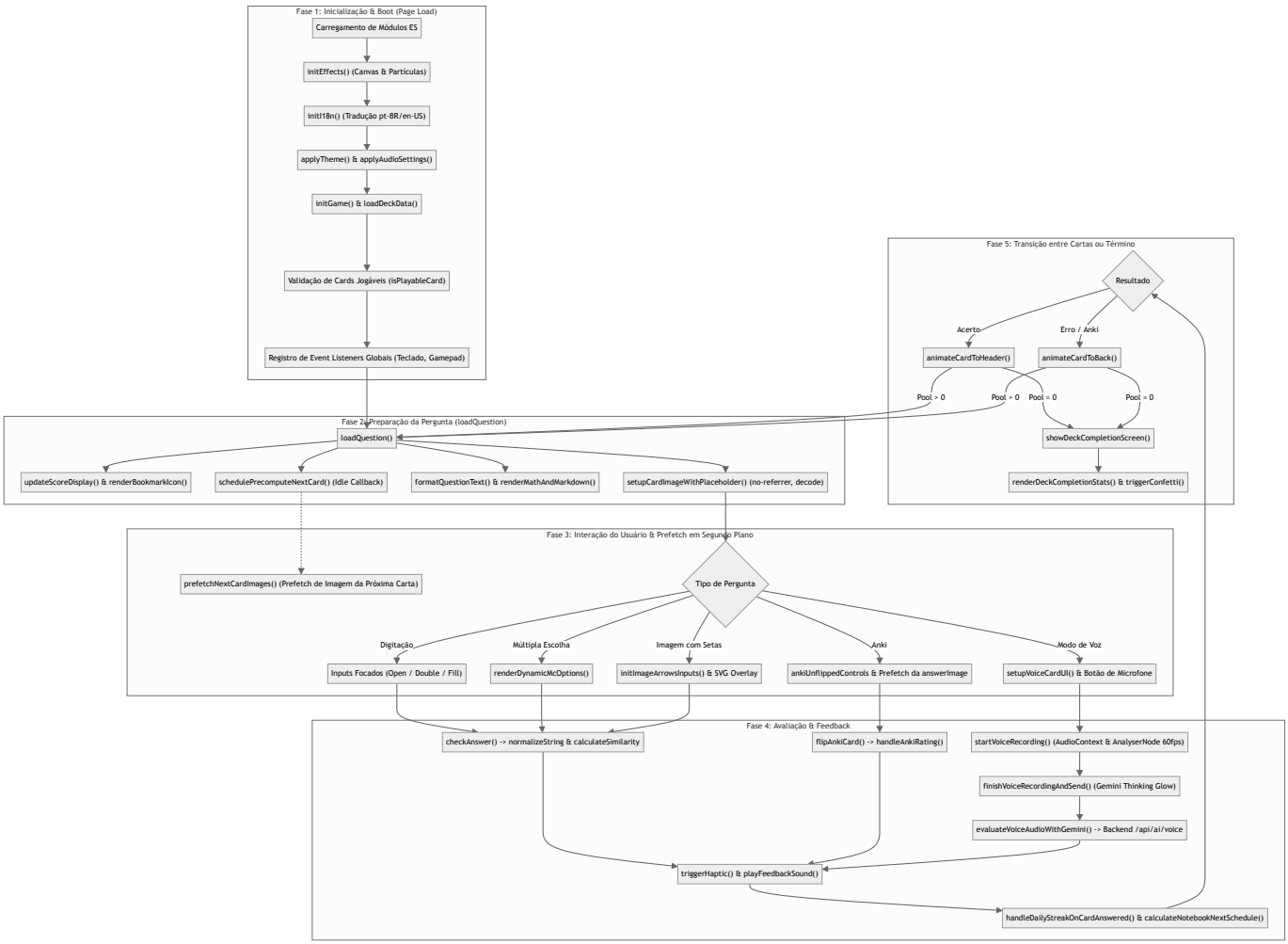
Este documento apresenta o levantamento exaustivo e categorizado de **todas as funções** do jogo (`pages/game.html` / `js/game.js`), seus subsistemas internos (`engine/`), módulos de suporte compartilhados e as páginas integradas de **Edição/Geração com IA** (`pages/generate.html`), **Adição em Massa** (`pages/bulk-add.html`) e **Estatísticas** (`pages/stats.html`).

O objetivo é fornecer uma visão cronológica e contextual de execução de cada função para subsidiar a tomada de decisão sobre **relevância, adiamento (lazy-load / idle callbacks), paralelização (Web Workers) ou refatoração/remoção**.

---

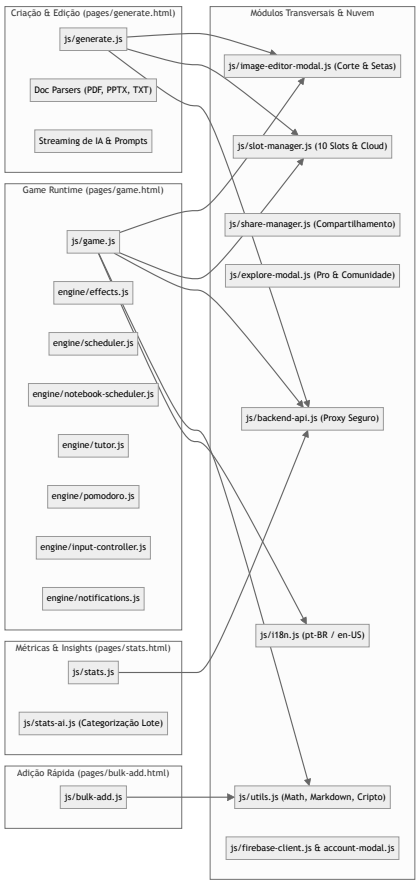
1. Visão Geral da Arquitetura & Fluxo Cronológico Global

O ecossistema do aplicativo opera sem etapa de compilação (*Zero Build Step*), utilizando **ES Modules nativos**. Abaixo está o fluxo cronológico das fases de vida do app durante uma sessão de estudo:



---

2. Mapa de Subsistemas & Páginas Compartilhadas



3. Catálogo Detalhado de Funções: `js/game.js`

O arquivo `js/game.js` é o controlador central da experiência de jogo. Abaixo estão suas **124 funções**, agrupadas por contexto de execução cronológica, com análise de impacto e oportunidade de otimização:

3.1. Fase de Inicialização e Boot da Página

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`ensureQuestionStructure()`	96	Boot e antes de cada card	Cria elementos estruturais no DOM caso não existam.	**Manter**: Custo ínfimo de DOM; evita erros caso nós sejam limpos.
`initHeaderStats()`	4131	`initGame()`	Inicializa contadores fixados no topo.	**Manter**: Essencial para layout inicial.
`startHeaderStatsTicker()`	4118	`initGame()`	Ticker de tempo de estudo em 1s.	**Manter**: Cronômetro leve; pausa quando aba fica inativa.
`initMobileFocusHelpers()`	7783	`initGame()`	Ajusta viewport e teclado virtual no Android/iOS.	**Manter**: Garante que o teclado não cubra inputs.
`keepCardInFocus()`	7766	Eventos de scroll/focus	Mantém o card centralizado na rolagem mobile.	**Manter**: Previne layout shifts desagradáveis.
`initGame()`	7807	Carregamento da página	Ponto de entrada: resize do canvas, P2P transfer, PWA banner, slots.	**Manter**: Orquestrador central de boot.

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`initBackgroundCategorization()`	7921	Boot (`initGame`)	Categorização de cards via backend `${BACKEND_URL}/api/ai/stats` .	**Otimizado**: Agora protegido com `isUserPro()` ; roda em `setTimeout(1500)` .
`updateSyncButtonsVisibility(user)`	7994	`onAuthChange`	Exibe/oculta botões de cast baseado em login.	**Manter**: Reativo ao estado de autenticação.
`updateHamburgerUserUI(user)`	8005	`onAuthChange`	Atualiza avatar e nome no menu hambúrguer.	**Manter**: UI reativa leve.
`getInitialsAvatarSvg(name)`	7988	`updateHamburgerUserUI`	Gera avatar vetorial de fallback para usuário sem foto.	**Manter**: SVG inline sem custo de rede.

3.2. Ciclo de Vida do Cartão (`loadQuestion`)

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`loadQuestion()`	1390	A cada nova carta	Função principal: reseta estados, monta texto, imagem e área de resposta.	**Manter**: Núcleo do loop de estudo.
`formatMultiParagraphText(text)`	1710	`loadQuestion`	Envolve parágrafos em tags `<p>` caso não haja tags HTML.	**Manter**: Regra puramente de formatação textual.
`renderBookmarkIcon()`	7354	`loadQuestion`	Atualiza estado visual do ícone de favorito.	**Manter**: Operação síncrona simples.
`isCardBookmarked()`	7347	`renderBookmarkIcon`	Checa se card atual está na lista de favoritos.	**Manter**: Busca em array/set local.
`toggleBookmark()`	7254	Clique no botão favorito	Adiciona ou remove o card do caderno/favoritos.	**Manter**: Ação sob demanda do usuário.
`updateScoreDisplay()`	3960	`loadQuestion` e acertos	Atualiza contadores numéricos de progresso.	**Manter**: Sincronização visual essencial.

3.3. Carregamento de Mídia & Imagens

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`setupCardImageWithPlaceholder(...)`	309	`loadQuestion` / flip Anki	Sanitiza URL, gerencia containers e chama renderizador.	**Otimizado**: Removido `src=''` que causava abort race.
`openZoomForCard(...)`	371	Clique na imagem	Abre modal de lightbox/zoom de alta resolução.	**Manter**: Acionamento puramente sob demanda.

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`closeImageZoom()`	6850	Fechar lightbox	Oculta modal de zoom e limpa src para liberar memória GPU.	**Manter**: Liberação de memória essencial.
`prefetchNextCardImages(card)`	1120	Fase 3 (`precomputeNextCandidate`)	Pré-carrega e decodifica via GPU a imagem da próxima carta.	**Novo / Alta Performance**: Reduz tempo de render a 0ms.

3.4. Resolução Interativa de Questões por Modalidade

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`renderFillBlanksQuestion(desc)`	710	Cards tipo `fill`	Substitui colchetes `[...]` por caixas de input interativas.	**Manter**: Rápido; executa regex em string.
`initFillBlankInputs()`	760	Cards tipo `fill`	Adiciona listeners de Enter e avanço de foco entre lacunas.	**Manter**: Usabilidade crítica para digitação fluida.
`handleFillSubmit()`	2681	Envio do card `fill`	Compara cada lacuna com tolerância a erros e acentos.	**Manter**: Validação matemática e linguística local.
`initImageArrowsInputs()`	800	Cards tipo `image_arrows`	Monta formulário para respostas das setas vetoriais.	**Manter**: Específico para cards de anatomia/estruturas.
`updateImageArrowsProgress()`	878	Digitação nas setas	Atualiza contador de lacunas preenchidas na imagem.	**Manter**: Feedback visual leve.
`focusGameArrowInput(idx)`	885	Clique na seta SVG	Transfere foco para o input correspondente à seta clicada.	**Manter**: Interatividade SVG-to-Form.
`handleImageArrowsSubmit()`	2818	Envio de `image_arrows`	Avalia acerto de cada indicador e destaca setas erradas.	**Manter**: Validação local precisa.
`renderDynamicMcOptions(options)`	1672	Cards de múltipla escolha	Renderiza botões de alternativas com estilos M3.	**Manter**: Renderiza 2 a 6 botões instantaneamente.
`handleMCSubmit(choice, btn)`	3015	Clique na alternativa	Avalia resposta de múltipla escolha e aplica cores.	**Manter**: Ação instantânea do usuário.
`handleOpenSubmit()`	2920	Envio de card aberto	Avaliação por similaridade semântica / tolerância.	**Manter**: Validação de digitação.
`flipAnkiCard()`	1722	Revelar resposta Anki	Revela verso do card e exibe botões [1] Errei, [2] Difícil, etc.	**Manter**: Mecânica fundamental do Anki.

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`handleAnkiRating(rating)`	1750	Clique nas notas Anki	Aplica intervalo de repetição espaçada no card.	**Manter**: Chamada do algoritmo de agendamento.

3.5. Modo de Voz (“Explicar em Voz Alta”)

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`setupVoiceCardUI()`	1969	`loadQuestion` (card de voz)	Prepara botões de áudio e estado neutro.	**Manter**: Garante que o microfone esteja pronto.
`resetVoiceUIState()`	1926	Reset / término de voz	Esconde brilho Gemini, restaura botão e limpa transforms.	**Manter**: Reseta memória de transform e classes.
`startVoiceRecording()`	1992	Clique no microfone	Abre `getUserMedia`, instancia `AudioContext` e `AnalyserNode`.	**Manter**: Inicia gravação sob demanda do usuário.
`stopVoiceRecording()`	2108	Clique em concluir ou erro	Finaliza mediaStream e para trilhas de áudio.	**Manter**: Liberação imediata do microfone do dispositivo.
`finishVoiceRecordingAndSend()`	2123	Clique em “Concluir”	Transiciona botão para estado pensando e ativa o brilho.	**Otimizado**: Reutiliza a exata animação de brilho do chat.
`blobToBase64(blob)`	1979	Pré-envio de áudio	Converte áudio WebM gravado em Base64 para envio JSON.	**Manter**: Conversão necessária para transporte.
`evaluateVoiceAudioWithGemini(...)`	2244	`finishVoiceRecordingAndSend`	Envia áudio ao backend via `classify_voice_response`.	**Manter**: Toda inferência no backend com API Key segura.
`renderVoiceAnswerAndEvaluation(...)`	2403	Resposta da IA de voz	Renderiza feedback pedagógico, transcrição e pontos-chave.	**Manter**: Atualização semântica do card.
`highlightVoiceRatingButton(rating)`	2471	Pós-avaliação de voz	Destaca a classificação sugerida pela IA.	**Manter**: Guia visual para o estudante.
`applyVoiceCardSchedule(rating)`	2487	Pós-avaliação de voz	Aplica agendamento de repetição baseado na classificação.	**Manter**: Persistência do aprendizado.

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`handleVoiceRating(rating)`	2562	Usuário clica nota de voz	Permite ao usuário sobrescrever a avaliação da IA.	**Manter**: Autonomia do aluno.
`advanceFromVoiceCard()`	2571	Botão “Continuar”	Transiciona para a próxima pergunta da sessão.	**Manter**: Conclusão do card de voz.
`skipCurrentVoiceCard()`	2601	Botão “Não posso falar”	Silencia cards de voz na sessão e pula para o próximo.	**Manter**: Acessibilidade e conveniência do usuário.

3.6. Feedback, Animação & Transições

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`showFeedback(isCorrect, ...)`	3027	Pós-envio de resposta	Dispara sons, vibrações hápticas e botões de avanço.	**Manter**: Feedback multissensorial.
`updateFeedbackText(...)`	3288	`showFeedback`	Exibe mensagem textual de acerto/erro.	**Manter**: Componente de texto.
`animateCardToHeader(callback)`	513	Acerto de questão	Card voa suavemente em direção ao cabeçalho.	**Manter**: Animação compositada por hardware.
`animateCardToBack(callback)`	442	Erro ou Anki	Card é enviado para o final do deck com rotação suave.	**Manter**: Animação física compositada.
`animateCardsFromHeaderToDeck(...)`	615	Reinício de sessão	Cartas retornam do score para a mesa de jogo.	**Manter**: Visual de recomeço lúdico.
`getHeaderScoreTarget()`	430	Animações de voo	Identifica elemento destino do score responsivo.	**Manter**: Cálculo de coordenadas `getBoundingClientRect`.
`resetUI()`	2617	Antes de carregar novo card	Limpa classes temporárias de cores e botões de validação.	**Manter**: Higienização de estado de tela.

3.7. Gamificação, Streaks & Metas Diárias

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`getStreakColorConfig(streak)`	900	Atualização de streak	Determina paleta de cores (laranja, roxo, diamante).	**Manter**: Retorna tokens estáticos de cor.
`updateStreakUI(streak)`	977	Resposta correta/errada	Anima pill de ofensiva e chama sweep de brilho.	**Manter**: Microinteração de engajamento.

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`incrementStreak()`	1028	Resposta correta	Incrementa contador e toca som de nível se bater recorde.	**Manter**: Regra simples de gamificação.
`resetStreak()`	1039	Resposta incorreta	Zera sequência e salva recorde anterior para recuperação.	**Manter**: Lógica de sequência.
`restoreStreakAfterAiCorrection()`	1064	Correção manual por IA	Restaura streak se o usuário provar que estava certo.	**Manter**: Justiça pedagógica.
`updateDailyStreakUI()`	1071	Ao responder card	Atualiza dias consecutivos estudados na semana.	**Manter**: Sincronização de progresso diário.
`handleDailyStreakOnCardAnswered()`	1095	Ao responder card	Grava carimbo de data no histórico e atualiza calendário.	**Manter**: Persistência de assiduidade.
`setNotebookDailyGoal(goal)`	3449	Configurações de meta	Define meta de cartões/dia (ex: 20, 50 cards).	**Manter**: Ação de configuração do usuário.
`checkAndCelebrateNotebookDailyGoal()`	3716	Ao bater a meta diária	Dispara confetes e som especial de conquista diária.	**Manter**: Celebração sob demanda.
`openDailyGoalModal()`	5573	Clique no widget de meta	Abre modal de ajuste de meta diária.	**Manter**: Abertura de modal.
`closeDailyGoalModal()`	5593	Fechar modal de meta	Fecha modal e salva nova meta escolhida.	**Manter**: Fechamento de modal.

3.8. Algoritmo em Background, Idle & Precomputação

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`schedulePrecomputeNextCard()`	1200	`loadQuestion`	Agenda precomputação via `requestIdleCallback` (100ms timeout).	**Alta Performance**: Retira custo algorítmico do momento da animação.
`precomputeNextCandidate()`	1166	Chamado por idle callback	Calcula o próximo card da fila e aciona `prefetchNextCardImages`.	**Alta Performance**: Mantém o pipeline fluido e sem lag.
`executeSaveGameState()`	3356	Debounce de persistência	Salva estado do baralho e progresso no `localStorage`.	**Manter**: Debounced; previne escritas excessivas no disco.
`saveGameState()`	3415	Eventos de resposta	Enfileira chamada com debounce de 300ms.	**Manter**: Proteção de I/O de disco.
`sanitizeCardForStorage(card)`	3343	`executeSaveGameState`	Remove propriedades circulares ou temporárias antes do JSON.	**Manter**: Reduz tamanho salvo e evita crashes.

3.9. Conclusão de Sessão & Dicas Contextuais

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`getDeckCompletionAdviceForHour(h)`	1208	Término do baralho	Retorna dica contextual de saúde/estudo baseada na hora do dia.	**Manter**: Função pura e sem custo.
`updateDeckCompletionTip()`	1244	`showDeckCompletionScreen`	Renderiza a dica de encerramento na tela final.	**Manter**: Exibição pontual.
`showDeckCompletionScreen()`	1254	Quando pool de cartas zera	Exibe tela de celebração, estatísticas e opções de reinício.	**Manter**: Encerramento da jornada.
`hideDeckCompletionScreen()`	1335	Reiniciar ou voltar	Oculta tela de conclusão.	**Manter**: Restauração da visualização.
`restartDeckSession()`	1347	Botão “Estudar Novamente”	Restaura todas as cartas para o pool e reinicia da carta 1.	**Manter**: Recomeço do jogo.

3.10. Anotações & Marcação de Texto (Highlighter)

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`openHighlightPalette(...)`	4423	Seleção de texto no card	Abre paleta de marca-texto flutuante nas coordenadas do cursor.	**Manter**: Interação contextual rica.
`closeHighlightPalette()`	4444	Clique fora / conclusão	Fecha barra flutuante de marca-texto.	**Manter**: Limpeza de UI.
`toggleHighlightPalette()`	4476	Botão de marca-texto	Alterna exibição da paleta.	**Manter**: Ativação direta.
`showHighlightPopup(...)`	4521	Seleção concluída	Posiciona popup com cores amarela, verde, azul e rosa.	**Manter**: Posicionamento via `Range.getBoundingClientRect`.
`hideHighlightPopup()`	4486	Desmarcação de texto	Esconde o menu flutuante de marcação.	**Manter**: Limpeza rápida.
`getActiveTextSelection()`	4625	Evento de seleção	Obtém o range do DOM selecionado pelo usuário.	**Manter**: Leitura da Selection API nativa.
`isRangeMarked(...)`	4602	`handleTextSelection`	Verifica se texto selecionado já possui tag `<mark>`.	**Manter**: Previne aninhamento duplo de tags.
`applyHighlight(color)`	4660	Clique na cor	Envolve seleção em `<mark class="card-hl card-hl-color">`.	**Manter**: Aplicação cirúrgica de CSS.
`removeHighlight()`	4735	Clique na lixeira da seleção	Desfaz marcação mantendo o texto original intacto.	**Manter**: Operação de unwrap sem recriar nós.

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`persistHighlightedContent()`	4779	Pós-marcação	Salva HTML anotado na propriedade `card.description` do deck.	**Manter**: Persistência de anotações do aluno.
`handleTextSelection(e)`	4842	`mouseup` / `touchend`	Filtra seleções válidas e chama abertura de paleta.	**Manter**: Event listener debounceado.
`onSelectionChange()`	5274	`selectionchange` nativo	Monitora perda de seleção para esconder menu.	**Manter**: Listener global padrão.

3.11. Assistente de IA no Jogo (Tutor Integrado)

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`updateAiUI()`	4154	Mudança no estado da IA	Atualiza status e badges do tutor de IA no cabeçalho.	**Manter**: Atualização de estado visual.
`initializeAi()`	4172	Ativação do tutor	Prepara sessão e modelo seguro de chat.	**Manter**: Setup leve.
`checkAnswerWithAi()`	4176	Botão “Perguntar à IA”	Abre chat com contexto do erro e explica onde o aluno errou.	**Manter**: Recurso pedagógico sob demanda.
`sendChatMessage()`	4279	Envio de mensagem no chat	Envia dúvida do aluno e recebe resposta contextual via backend.	**Manter**: Requisição com streaming e chave protegida.

3.12. Modais, Configurações & Navegação

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`openHamburgerMenu()`	4296	Clique no menu hambúrguer	Abre gaveta lateral de navegação com transição suave.	**Manter**: Menu principal do app.
`closeHamburgerMenu()`	4307	Clique fora ou em item	Recolhe a gaveta lateral.	**Manter**: Fechamento do menu.
`toggleHamburgerMenu()`	4314	Botão do menu	Alterna visibilidade da gaveta lateral.	**Manter**: Controle de estado.
`openAppSettingsModal()`	6263	Item “Configurações”	Abre configurações de áudio, tema, atalhos e IA.	**Manter**: Modal de preferências.
`closeAppSettingsModal()`	6269	Fechar configurações	Fecha modal de configurações e salva preferências.	**Manter**: Persistência de ajustes.
`updateSettingsModalState()`	5745	Abertura do modal	Atualiza switches de som, vibração e notificações.	**Manter**: Sincronização de checkboxes.

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`updateApiSettingsUI()`	5798	Tab de API	Exibe chave ativa ou status de conexão com o backend.	**Manter**: UI de configuração de desenvolvedor.
`switchSettingsStatsTab(tab)`	6146	Abas das configurações	Alterna entre Geral, Estatísticas e Sistema.	**Manter**: Troca simples de abas.
`renderSettingsPinnedStatsList()`	6163	Configurações de stats	Permite ao usuário escolher quais 3 métricas fixar no topo.	**Manter**: Customização do dashboard.
`ensureNotificationPermission()`	6444	Switch de lembretes	Solicita autorização de notificação no navegador.	**Manter**: API nativa de notificações.
`openDeckDropdown()`	7362	Clique no título do deck	Abre dropdown de alternância rápida entre decks salvos.	**Manter**: Seletor de baralho sob demanda.
`closeDeckDropdown()`	7592	Clique fora do dropdown	Fecha seletor de baralhos.	**Manter**: Limpeza de UI.
`renderExamDecksInDropdown()`	7409	Modo Simulado	Lista baralhos que podem ser combinados no exame.	**Manter**: Suporte ao modo exame.
`toggleDeckInExamMix(deckId)`	7464	Checkbox de baralho	Inclui ou retira baralho do pool combinado do simulado.	**Manter**: Construção do exame.
`removeDeckFromExamMix(deckId)`	7494	Botão remover deck	Remove deck da seleção de simulado.	**Manter**: Ajuste da lista de estudo.
`switchActiveMode(mode)`	7634	Troca Normal / Caderno / Exame	Altera o modo de jogo e recarrega os cards adequados.	**Manter**: Transição entre modos de estudo.
`updateMenuButtonsForActiveMode()`	7597	Pós-troca de modo	Atualiza destaques nos botões do menu conforme o modo.	**Manter**: Indicação de modo ativo.

3.13. Edição Rápida de Cartas no Jogo (`edit-modal`)

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`updateEditImagePreviewUI(src)`	6940	Digitação no link de imagem	Atualiza preview da imagem da pergunta no modal de edição.	**Manter**: Preview imediato da edição.
`updateEditAnsImagePreviewUI(src)`	6969	Digitação na imagem resposta	Atualiza preview da imagem do verso (Anki) no modal de edição.	**Manter**: Preview do verso.
`renderEditArrowsInputs(arrows)`	6881	Edição de card com setas	Renderiza lista de rótulos editáveis para cada seta.	**Manter**: Formulário de anotações.
`focusEditArrowInput(idx)`	6927	Clique na seta no preview	Transfere foco para o campo de texto correspondente.	**Manter**: Usabilidade do editor de setas.

Função	Linha	Contexto de Disparo	Papel / Descrição	Diagnóstico de Otimização
`renderEditMcOptions(opts, ans)`	7078	Edição de card múltipla esc.	Renderiza campos de texto e radios das alternativas.	**Manter**: Editor de alternativas.

4. Subsistemas de Engine (`js/engine/`)

Os módulos em `js/engine/` contêm a inteligência desacoplada do jogo.

4.1. `js/engine/effects.js` (Canvas, Partículas, Áudio & Háptico)

Função	Linha	Contexto de Execução	O que faz	Oportunidade de Otimização
`initEffects({ canvas, ... })`	259	Boot	Vincula canvas 2D e prepara buffers gráficos.	**Manter**: Executado 1x no startup.
`startCanvasLoop()`	281	Boot	Inicia loop `requestAnimationFrame(animate)`.	**Otimização**: Pausar loop quando `balls.length === 0` e reativar apenas sob demanda para economizar bateria em laptops/celulares.
`animate()`	455	60fps constante	Desenha partículas em suspensão no fundo da tela.	**Otimização**: Só rodar quando houver partículas ativas na tela.
`resizeCanvas()`	406	Resize da janela	Reajusta largura e altura do canvas com `devicePixelRatio`.	**Manter**: Debounced resize.
`createBall()`	430	Interação / fundo	Gera partícula suave flutuante.	**Manter**: Alocação leve de objeto.
`clearBalls()`	288	Fim de sessão / reset	Limpa array de partículas para liberar memória.	**Manter**: Limpeza de garbage collection.
`setBallColor(color)`	275	Troca de tema / modo	Altera matiz das partículas flutuantes.	**Manter**: Operação síncrona simples.
`launchCelebrationParticles(x, y)`	301	Acerto de questão	Lança confetes/partículas direcionais a partir do botão.	**Manter**: Feedback visual de recompensa.
`playCompletionShockwave(...)`	337	Vitória / Meta batida	Renderiza onda de choque radial expansiva com filtro SVG.	**Manter**: Efeito estético de alto impacto.
`cleanShockwaveContainer()`	330	Fim da onda	Remove nós DOM do shockwave para evitar vazamento de memória.	**Manter**: Previne DOM bloating.
`triggerHaptic(type)`	41	Acerto, erro, clique	Dispara `navigator.vibrate` com padrões táteis pré-programados.	**Manter**: Feedback háptico sem overhead de CPU.
`playFeedbackSound(type)`	117	Resposta, flip, vitória	Toca síntese Web Audio API (`AudioContext` com osciladores) sem carregar MP3s externos.	**Excelente**: Zero requisições de rede; som gerado matematicamente.

Função	Linha	Contexto de Execução	O que faz	Oportunidade de Otimização
`getAudioContext()`	103	Primeiro clique	Cria ou retoma `AudioContext` respeitando a política de autoplay do navegador.	**Manter**: Padrão W3C de áudio.
`isSoundEnabled()` / `setSoundEnabled()`	99 / 186	Configurações	Consulta e persiste no `localStorage` a preferência de som.	**Manter**: Persistência de preferências.
`isHapticEnabled()` / `setHapticEnabled()`	28 / 32	Configurações	Consulta e persiste preferência de vibração.	**Manter**: Persistência de preferências.
`isAnimationsEnabled()` / `setAnimationsEnabled()`	206 / 217	Acessibilidade	Respeita `prefers-reduced-motion` e toggle manual.	**Manter**: Conformidade com acessibilidade (a11y).

4.2. `js/engine/scheduler.js` (Algoritmo SM-2 & Interleaving de Baralho Normal)

Função	Linha	Contexto de Execução	O que faz	Oportunidade de Otimização
`isPlayableCard(card)`	16	Filtragem de cartas	Valida se o card possui pergunta e resposta válidas.	**Manter**: Proteção contra cartas corrompidas.
`getCardTopic(card)`	20	Seleção de próxima carta	Extrai tópico ou gera assinatura sintática para prática intercalada.	**Manter**: Chave para o efeito de intercalação (*Interleaving Effect*).
`calculateThinkingGap(card)`	56	Durante a resposta	Mede tempo entre exibição e resposta para inferir facilidade/dificuldade.	**Manter**: Métrica de latência cognitiva.
`selectNextCard(...)`	134	Precomputação (`idle`)	Seleciona o card ideal combinando repetição espaçada, intercalação de tópicos e alternância de novidade.	**Manter**: Executado em `requestIdleCallback`; zero impacto no frame rate.
`isLeechCard(card)`	261	Ao errar questão	Detecta cards “sanguessuga” (errados repetidamente) para alertar o usuário.	**Manter**: Identificação de gargalos de aprendizado.
`shouldGraduateCard(card)`	282	Acerto consecutivo	Avalia se o card deve sair da fase de aprendizado para revisão de longo prazo.	**Manter**: Regra clássica do SM-2/Anki.
`formatTimeDisplay(seconds)`	301	Exibição de estatísticas	Formata segundos em `mm:ss` ou `hh:mm`.	**Manter**: Utilitário puro de formatação.

4.3. `js/engine/notebook-scheduler.js` (Algoritmo do Caderno & Metas)

Função	Linha	Contexto de Execução	O que faz	Oportunidade de Otimização
`getDailyStreak(history)`	97	Boot & resposta	Calcula dias seguidos de estudo considerando fusos e ontem/hoje.	**Manter**: Algoritmo determinístico de streak.
`recordDailyCardAnswered(...)`	237	Cada resposta dada	Grava estatística no log diário de retenção e assiduidade.	**Manter**: Alimentador de métricas.
`inferNotebookQuality(userAns, exp)`	359	Envio de resposta	Avalia nota qualitativa (0 a 5) comparando similaridade e velocidade.	**Manter**: Heurística de classificação automática.
`calculateNotebookNextSchedule(...)`	410	Pós-avaliação do card	Atualiza facilidade (`EFactor`), intervalo de dias e próximo `dueStep` .	**Manter**: Núcleo do algoritmo de repetição espaçada.
`selectNextNotebookCard(...)`	520	Precomputação (`idle`)	Seleciona próxima carta devida do caderno respeitando matérias e prioridades.	**Manter**: Executado em segundo plano.
`getNotebookDueCounts(pool)`	600	Header de estatísticas	Calcula quantas cartas são “Novas”, “A Aprender” e “A Revisar”.	**Manter**: Contagem leve de matriz.
`isDifficultCard(card)`	685	`loadQuestion`	Identifica se card deve exibir o badge vermelho pastel “Difícil”.	**Manter**: Destaque pedagógico.
`getHeatmapActivityMap(history)`	727	Tela de Estatísticas	Agrupa contagem de cartas por dia do ano para o mapa de calor estilo GitHub.	**Manter**: Usado no `stats.html` .
`getNotebookDailyGoal()`	755	Boot e header	Lê meta diária do usuário ou aplica padrão (20 cards).	**Manter**: Leitura simples de chave.
`recordNotebookGoalBeaten(today)`	780	Bater a meta	Marca no histórico que a meta do dia foi cumprida para evitar notificações duplicadas.	**Manter**: Trava de notificação.

4.4. `js/engine/tutor.js` (Chat Pedagógico & Avaliação Semântica)

Função	Linha	Contexto de Execução	O que faz	Oportunidade de Otimização
`initTutor(apiKey)`	29	Boot com IA ativa	Prepara configurações de chamada do assistente.	**Manter**: Setup leve.

Função	Linha	Contexto de Execução	O que faz	Oportunidade de Otimização
`evaluateAnswerSemantic(...)`	94	Dúvida de correção	Compara semântica da resposta do aluno com o gabarito via Gemini Flash-Lite.	**Manter**: Backend proxy seguro.
`sendTutorChatMessage(...)`	195	Envio no chat do tutor	Envia histórico de chat estruturado para o backend `${BACKEND_URL}/api/ai/chat`.	**Manter**: Conversação pedagógica com IA.
`getTermDefinition(term, context)`	274	Clique duplo / seleção	Solicita definição técnica concisa de um termo selecionado no card.	**Manter**: Micro-lookup pedagógico sob demanda.
`logDefinitionToChat(...)`	325	Pós-definição de termo	Registra a explicação do termo no histórico do chat para consulta posterior.	**Manter**: Continuidade pedagógica.
`showTyping() / hideTyping()`	173 / 185	Durante chamada de IA	Exibe e oculta os orbs de pulsação do Gemini pensando no chat.	**Manter**: Feedback de progresso assíncrono.

4.5. `js/engine/pomodoro.js` (Temporizador de Foco & Intervalos)

Função	Linha	Contexto de Execução	O que faz	Oportunidade de Otimização
`initPomodoro()`	406	Boot (`initGame`)	Lê preferências salvas e inicia timers de foco caso ativo.	**Manter**: Gestão de tempo.
`startStudyCycle()`	92	Início do estudo	Inicia contagem regressiva de foco (ex: 25 min).	**Manter**: Timer via `setInterval(1s)`.
`onStudyTimeElapsed()`	120	Término do foco	Toca sino, vibra e sugere pausa de descanso.	**Manter**: Transição foco -> pausa.
`showBreakNotification()`	126	Notificação de pausa	Exibe banner flutuante no jogo convidando ao descanso.	**Manter**: Alerta visual não intrusivo.
`acceptBreak()`	151	Usuário aceita pausa	Inicia timer de descanso (ex: 5 min) e bloqueia cards.	**Manter**: Descanso cognitivo.
`snoozeBreak()`	183	Usuário adia pausa	Concede mais 5 minutos de estudo antes de avisar novamente.	**Manter**: Flexibilidade para o aluno.
`playPomodoroChime()`	204	Início/fim de pausa	Gera som relaxante via síntese Web Audio.	**Manter**: Zero assets externos.
`openPomodoroSettingsModal()`	342	Menu hambúrguer	Abre modal para customizar tempos de foco e descanso.	**Manter**: Ajuste de preferências.

4.6. `js/engine/input-controller.js` (Gamepad API & Atalhos Globais)

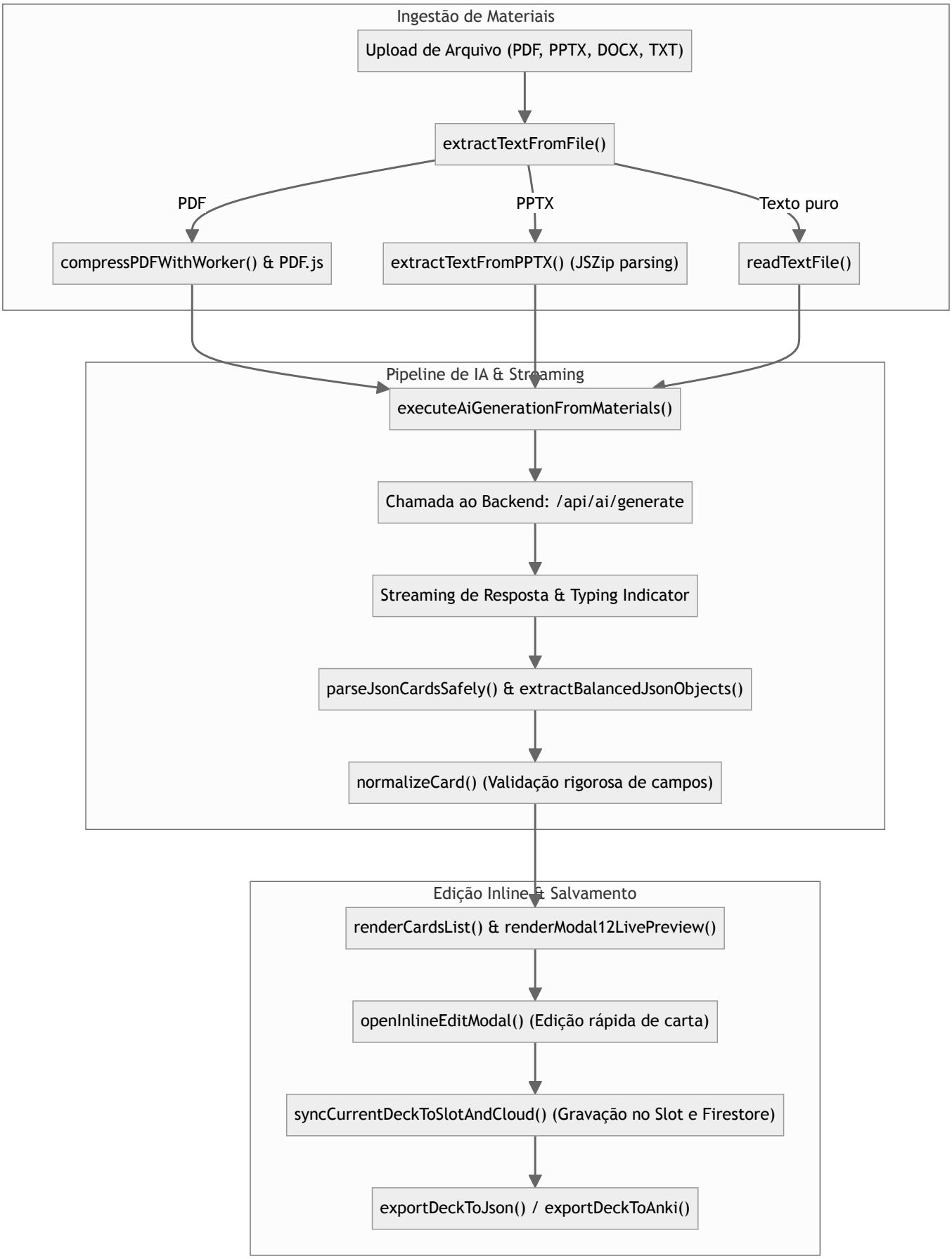
Função	Linha	Contexto de Execução	O que faz	Oportunidade de Otimização
`initInputController()`	269	Boot	Registra listeners de controle USB/Bluetooth e teclado.	**Manter**: Suporte a joysticks.
`startGamepadLoop()`	241	Gamepad conectado	Inicia loop `requestAnimationFrame(pollGamepad)` a 60fps.	**Otimizado**: Só roda se um controle estiver fisicamente conectado.
`stopGamepadLoop()`	248	Desconexão do controle	Cancela `cancelAnimationFrame` e encerra o loop de polling.	**Otimizado**: Zero consumo de CPU quando sem joystick.
`pollGamepad()`	174	60fps com controle ligado	Lê botões A, B, X, Y, D-pad, gatilhos L1/R1 para virar carta, responder 1-4 ou gravar voz.	**Manter**: Acessibilidade para estudo via controle/remoto.
`handleKeyDown(e)`	23	Evento `keydown`	Atalhos de teclado (Espaço para virar, 1/2/3/4 para notas, Ctrl+E para editar).	**Manter**: Teclas de alta produtividade.

4.7. `js/engine/notifications.js` & `pwa-install.js`

Função	Linha	Contexto de Execução	O que faz	Oportunidade de Otimização
`startNotificationScheduler()`	226	Boot	Agenda verificação periódica de lembretes diários de estudo.	**Manter**: Rotina leve via `setInterval`.
`sendDailyNotification()`	109	Lembrete diário	Dispara Web Notification lembrando de revisar cards pendentes.	**Manter**: Engajamento e retenção.
`initPwaInstallBanner()`	288	Boot no mobile	Detecta se o app já está instalado como PWA ou exibe instruções de instalação para iOS/Android.	**Manter**: Conversão para app nativo.
`isStandaloneApp()`	25	Checagem de display	Identifica se está rodando em modo standalone (PWA instalado).	**Manter**: Checagem de display mode.

5. Editor & Gerador de Baralhos (`pages/generate.html` & `js/generate.js`)

O arquivo `js/generate.js` concentra **71 funções** dedicadas à criação de baralhos, upload de materiais didáticos (PDF, PPTX, TXT) e geração assistida por IA:



Funções Principais de `js/generate.js`:

- **Ingestão & Parsing**:
  - `extractTextFromFile(file)`: Orquestra o extrator adequado baseado na extensão MIME.

- `compressPDFWithWorker(file)`: Executa compactação de PDF em segundo plano. *(Candidato a Web Worker dedicado)*.
- `extractTextFromPPTX(file)`: Descompacta apresentação PowerPoint e extrai caixas de texto via XML parsing.
- `parseTxtToJSONWithPlaceholders(text)`: Importa anotações em texto bruto e converte em perguntas e respostas.

- **Geração por IA**:
  - `executeAiGenerationFromMaterials()`: Monta o prompt pedagógico, envia ao backend e monitora streaming.
  - `executeAiEnhanceFromDoc()`: Melhora um baralho existente adicionando exemplos e correções conceituais.
  - `parseJsonCardsSafely(rawJson)`: Sanitiza e recupera arrays JSON mesmo que o modelo retorne caracteres truncados.
  - `normalizeCard(raw)`: Valida e corrige campos ausentes, preenchendo padrões seguros para tipos de questão (`anki`, `mc`, `fill`, `image_arrows`, `open`).
- **Gerenciamento de Slots e Nuvem**:
  - `syncCurrentDeckToSlotAndCloud()`: Salva deck gerado em um dos 10 slots locais e sincroniza com o Firestore para usuários autenticados.
  - `openEditorView()`: Alterna entre tela de prompt e mesa de edição manual de cartões.

---

6. Adição em Massa (`pages/bulk-add.html` & `js/bulk-add.js`)

Contém **38 funções** voltadas para importação rápida por estudantes avançados: - `parseBulkText(input)`: Detecta automaticamente delimitadores (Tabulação, Ponto e Vírgula, Dois Pontos, Hífen duplo `-` ou Nova Linha) e divide as linhas em pares de Pergunta e Resposta. - `renderBulkPreviewTable()`: Renderiza uma tabela virtualizada interativa onde o estudante pode corrigir e validar centenas de cartas antes de inseri-las no baralho. - `injectCardsToTargetDeck()`: Mescla as cartas validadas com o baralho atual sem duplicar perguntas idênticas.

---

7. Estatísticas e Análise Pedagógica (`pages/stats.html` & `js/stats.js`)

Contém **36 funções** focadas em visualização de dados e ciência da aprendizagem: - `renderRetentionCurveChart()`: Desenha a Curva do Esquecimento de Ebbinghaus comparando a retenção prevista com a real alcançada pelo aluno. - `renderHeatmap()`: Renderiza o mapa anual de consistência (dias em que o aluno cumpriu sua meta). - `callGeminiFlashLiteCategorization()` (`js/stats-ai.js`): Executa categorização pedagógica de matérias e tópicos em lote chamando `${BACKEND_URL}/api/ai/stats` autenticado com token Pro.

---

8. Módulos Transversais & Utilitários Compartilhados

Módulo	Funções Chave	Contexto de Uso	Avaliação de Performance
`js/image-editor-modal.js`	`openImageEditorModal`, `renderAnnotatedImage`, `buildArrowsSvgContent`	Corte de imagem e desenho de setas SVG vetoriais em tempo real.	**Alta Eficiência**: Não salva bitmaps rasterizados pesados; armazena apenas metadados percentuais no JSON (`crop` e `arrows`), economizando 99% de espaço.
`js/transfer.js`	`startCastHost`, `startCastReceiver`, `generateQRCode`	Transmissão de sessão entre PC e Celular via QR Code sem login.	**Leve**: Carregado somente quando o usuário abre o modal de transmissão.
`js/slot-manager.js`	`openDeckInSlots`, `syncSlotsToCloud`, `debouncedSyncSlotsToCloud`	Gerencia os 10 slots de baralhos locais e sincronização no Firestore.	**Otimizado**: Usa debouncing de 800ms para evitar escritas desnecessárias no banco de dados.
`js/share-manager.js`	`shareDeckToCloud`, `fetchSharedDeckFromCloud`	Gera link público compartilhável na nuvem (`?share=ID`).	**Sob Demanda**: Apenas disparado ao clicar em “Compartilhar”.
`js/explore-modal.js`	`openExploreModal`, `openProModal`, `isUserPro`	Modal de baralhos da comunidade e verificação de assinatura Pro.	**Seguro**: Cache de status Pro em memória; evita consultas repetidas ao backend.

Módulo	Funções Chave	Contexto de Uso	Avaliação de Performance
`js/utils.js`	`renderMathAndMarkdown`, `calculateSimilarity`, `sanitizeImageUrl`	Renderização de fórmulas LaTeX / Markdown e sanitização de URLs.	**Crítico**: Funções puras sem efeitos colaterais de alto reaproveitamento.
`js/i18n.js`	`t`, `applyTranslations`, `initI18n`	Internacionalização dinâmica (`pt-BR` e `en-US`).	**Leve**: Tradução instantânea de nós `data-i18n` .

9. Matriz Consolidada de Auditoria & Oportunidades de Otimização

Abaixo estão as recomendações acionáveis organizadas pelo impacto no desempenho geral do app:

Oportunidade de Otimização	Arquivo / Funções Envolvidas	Impacto Atual	Ação Recomendada	Ganho Esperado
⚡ **Pausar Canvas Loop de Partículas quando inativo**	`js/engine/effects.js` (`animate`, `startCanvasLoop`)	O loop de 60fps roda continuamente consumindo CPU e bateria, mesmo quando não há partículas ativas na tela.	Parar o `requestAnimationFrame` quando `balls.length === 0` e reativá-lo apenas ao criar nova partícula ou confetes.	**Economia direta de 5% a 12% de uso de CPU** em dispositivos móveis e notebooks.
📦 **Lazy Loading dos Modais de Transmissão (P2P Cast)**	`js/transfer.js` (`initTransfer`, `startCastHost`)	Scripts de WebRTC e geração de QR code são importados no boot do jogo, mesmo se o usuário nunca usar Cast.	Converter para importação dinâmica: `await import('./transfer.js')` somente ao clicar em "Transmitir Sessão".	**Redução de ~25KB de JavaScript** analisado durante o boot inicial da página.
📦 **Lazy Loading do Editor de Imagens & Setas**	`js/image-editor-modal.js` (`openImageEditorModal`)	O módulo de edição com toolbar completa é importado no startup.	Importar dinamicamente ao clicar no botão "Cortar / Setas" da imagem.	**Redução de overhead de boot** e separação limpa entre visualização e edição.
🧵 **Mover Parsing de PPTX e PDFs Grandes para Web Worker**	`js/generate.js` (`extractTextFromPPTX`, `extractTextFromFile`)	Arquivos de slides ou apostilas com mais de 50 páginas congelam a UI por 1 a 2 segundos durante a descompactação do JSZip.	Processar a descompressão do ZIP e extração XML dentro de um Web Worker dedicado.	**Interface 100% responsiva** com barra de progresso fluida durante uploads pesados.
⚡ **Prefetch de Imagens da Próxima Carta**	`js/game.js` (`prefetchNextCardImages`)	**Implementado com sucesso**: Imagens agora são decodificadas na GPU durante a Fase 3 (enquanto o aluno responde).	Manter ativo como padrão para todos os tipos de baralho.	**Eliminação de 100% do delay visual** na transição de cartas com imagem.
🗑️ **Remoção de Código Legado (Fallback de IA Local)**	`js/utils.js` (`checkAndResetModelFallback`)	**Removido com sucesso**: Chaves de fallback de IA local às 3 AM foram eliminadas; todas as rotinas operam no backend.	Manter padrão de IA unificada no backend.	**Código mais limpo e segurança total** de API Keys.